تقرير النشاط العملي: التحليل التطبيقي لأداء خوارزميات الترتيب

تراكيب البيانات والخوارزميات (Data Structures & Algorithms) — لغة Java

1. البيانات الشخصية

اسم الطالب	يوسف سامي ابراهيم
الرقم الجامعي	3166027
رقم الشعبة	قروب A

2. وصف التجربة وبيئة التنفيذ

تم تنفيذ برنامج بلغة Java يقيس زمن تنفيذ خمس خوارزميات ترتيب هي: Selection Sort وInsertion Sort وBubble Sort وMerge Sort وQuick Sort، وذلك على مصفوفات من الأعداد الصحيحة حجمها ثابت N = 100,000 عنصر، وعلى أربع حالات للبيانات:

- **غير مرتبة (Random):** أعداد عشوائية بين 0 و999,999.
- **مرتبة عكسيًا:** عناصر مرتبة من الأكبر إلى الأصغر.
- **مرتبة جزئيًا:** مصفوفة مرتبة تصاعديًا ثم استُبدل نحو 20% من مواضعها (مواضع عشوائية متفرقة) بقيم عشوائية، فتبقى مرتبة بنسبة تقارب 80%.
- **مرتبة تمامًا:** عناصر مرتبة تصاعديًا.

منهجية القياس:

- استُخدمت System.nanoTime() وقُسمت النتيجة على 1000 للحصول على الزمن بالميكروثانية (µs).
- أُخذت نسخة مستقلة ومطابقة من المصفوفة الأصلية قبل كل خوارزمية، فكل خوارزمية تعمل على البيانات نفسها تمامًا.
- قِيس زمن عملية الترتيب فقط، واستُبعد زمن توليد المصفوفات ونسخها، كما تم التحقق من صحة الترتيب بعد انتهاء القياس.
- أُجري تسخين قصير لمترجم JIT على مصفوفة صغيرة قبل القياسات الفعلية، وثُبّتت بذرة المولّد العشوائي (Seed = 42) ليتم الاختبار على البيانات نفسها في كل تشغيل.
- اعتُمد في Quick Sort العنصر الأخير كعنصر محور (Pivot)، ونُفّذ البرنامج في مسار (Thread) بحجم مكدس كبير لأن الاستدعاء الذاتي قد يصل عمقه إلى 100,000 مستوى في الحالات السيئة.

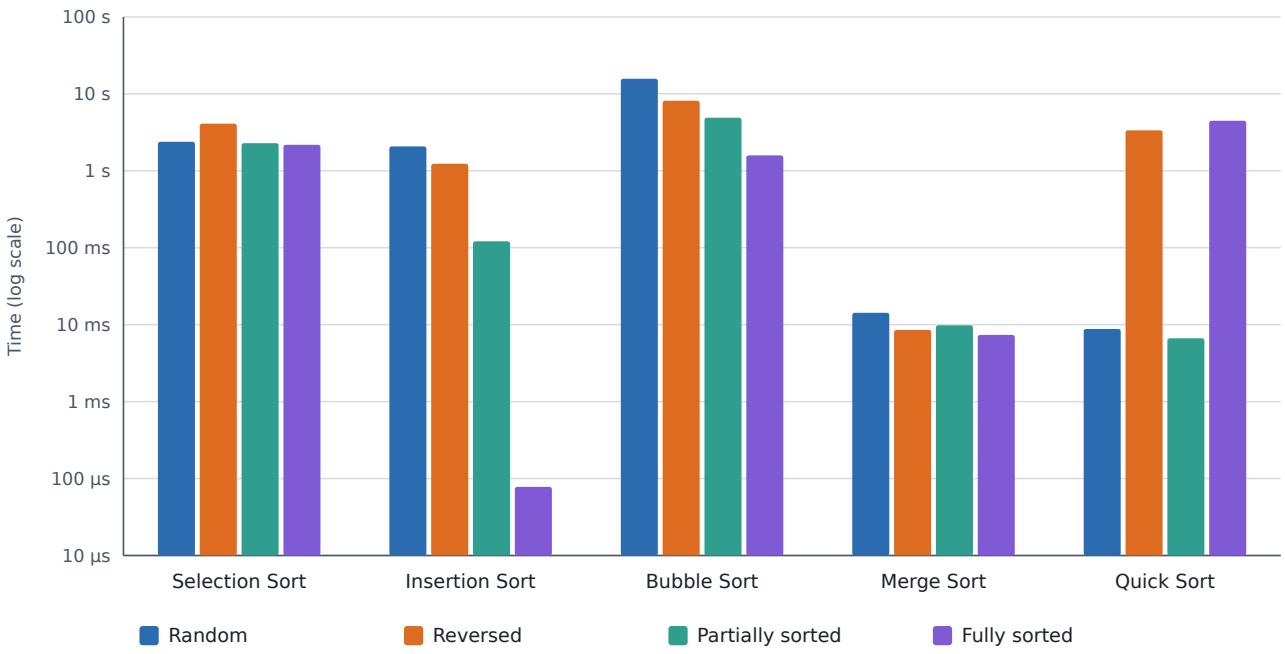
- بيئة التنفيذ: نظام Linux، وOpenJDK 21.

3. جدول نتائج الأزمنة (بالميكروثانية)

الخوارزمية	غير مرتبة (Random)	مرتبة عكسيًا	مرتبة جزئيًا	مرتبة تمامًا
Selection Sort	2,380,258 µs	4,089,815 µs	2,284,766 µs	2,178,647 µs
Insertion Sort	2,069,520 µs	1,235,520 µs	121,116 µs	**78 µs**
Bubble Sort	15,723,186 µs	8,120,739 µs	4,882,981 µs	1,584,707 µs
Merge Sort	14,245 µs	**8,522 µs**	9,815 µs	7,371 µs
Quick Sort	**8,777 µs**	3,356,974 µs	**6,639 µs**	4,464,168 µs

الخلايا المظللة بالأخضر هي أسرع زمن في كل حالة. للتوضيح: 1,000,000 µs = ثانية واحدة. وتعتمد القيم المطلقة على جهاز التشغيل، أما الفروقات النسبية بين الخوارزميات فهي المقصودة بالمقارنة.

4. الرسم البياني للمقارنة



الشكل (1): مقارنة أزمنة الخوارزميات في الحالات الأربع (مقياس لوغاريتمي)

استُخدم المقياس اللوغاريتمي للمحور الرأسي لأن الفرق بين أسرع زمن (78 µs) وأبطأ زمن (أكثر من 15 مليون µs) يبلغ عدة مراتب عشرية، ولا يمكن عرضه بمقياس خطي بوضوح.

5. تحليل ومناقشة النتائج

5.1 مقارنة خوارزميات $O(N^2)$ مع خوارزميات $O(N \log N)$

خوارزميات Selection وInsertion وBubble تنتمي إلى الفئة التربيعية $O(N^2)$، أما Merge وQuick فتنتمي إلى الفئة $O(N \log N)$. عند $N = 100{,}000$ تعني الفئة التربيعية قرابة عشرة مليارات عملية في الحالة العامة، مقابل نحو 1.7 مليون عملية فقط $(N \cdot \log_2 N)$ للفئة اللوغاريتمية. وهذا ما ظهر في النتائج على البيانات العشوائية:

- Merge Sort أسرع من Selection Sort بنحو **167** مرة، ومن Insertion Sort بنحو **145** مرة، ومن Bubble Sort بنحو **1,104** مرة.
- Quick Sort أسرع من Selection Sort بنحو **271** مرة، ومن Bubble Sort بنحو **1,791** مرة.
- تحتاج Merge وQuick إلى أجزاء من الألف من الثانية (14.2 ms و8.8 ms)، بينما تحتاج الخوارزميات التربيعية إلى ثوانٍ (2.38 s و2.07 s و15.72 s).
- تضاعف حجم البيانات يضاعف زمن الخوارزميات التربيعية أربع مرات تقريبًا، لكنه يضاعف زمن الخوارزميات اللوغاريتمية قليلًا فوق الضعف، ولهذا يتسع الفرق كلما زاد الحجم.
- Bubble Sort كانت الأبطأ بين الخوارزميات التربيعية في الحالة العشوائية لأنها تُجري عددًا كبيرًا من عمليات التبديل (Swaps)، في حين تُجري Selection Sort تبديلًا واحدًا فقط في كل دورة خارجية.

5.2 الخوارزمية الأفضل مع البيانات المرتبة والمرتبة جزئيًا

البيانات المرتبة تمامًا: الأفضل هي **Insertion Sort** بزمن 78 µs فقط. السبب أن حلقتها الداخلية لا تنفّذ أي إزاحة عندما يكون كل عنصر أكبر من سابقه، فتجري مقارنة واحدة لكل عنصر ويصبح التعقيد خطيًا $O(N)$ في أفضل حالاتها. وهي أسرع هنا بنحو 94 مرة من Merge Sort (7,371 µs) التي تبقى $O(N \log N)$ مهما كانت البيانات لأنها تقسّم وتدمج دائمًا.

البيانات المرتبة جزئيًا: كان أسرع زمن مقيس لـ **Quick Sort** (6,639 µs) تليها Merge Sort (9,815 µs)، والفرق بينهما صغير ومن الطبيعي أن يتبدل ترتيبهما بين تشغيل وآخر. أما Insertion Sort (121,116 µs) فهي الأفضل بين الخوارزميات التربيعية بفارق كبير، وأسرع من حالتها العشوائية بنحو 17 مرة، لأن عدد الإزاحات فيها يساوي عدد المقلوبات (Inversions) في المصفوفة، وهو قليل عندما تكون معظم العناصر في أماكنها. أما Selection Sort فلم تستفد إطلاقًا (2.28 s)، لأنها تفحص بقية المصفوفة كاملة في كل دورة مهما كان ترتيبها.

الخلاصة: Insertion Sort هي الخيار الأنسب للبيانات المرتبة أو شبه المرتبة وحجمها صغير أو متوسط، بينما تبقى Merge Sort وQuick Sort الخيار المناسب للأحجام الكبيرة في الحالة العامة.

5.3 سلوك Quick Sort في الحالة العكسية والمرتبة تمامًا وتأثير عنصر المحور

سجلت Quick Sort أسوأ نتائجها في هاتين الحالتين: 3.36 s في العكسية و4.46 s في المرتبة تمامًا، مقابل 8.8 ms في الحالة العشوائية، أي أبطأ بنحو 382 و509 مرة على الترتيب، لتتحول من

O(N log N) إلى **O(N²)**.

التفسير يعود إلى اختيار المحور: اعتُمد العنصر الأخير كمحور. في المصفوفة المرتبة تصاعديًا يكون العنصر الأخير هو الأكبر دائمًا، وفي المرتبة عكسيًا يكون الأصغر دائمًا. عندئذ تنقسم المصفوفة عند كل استدعاء إلى جزء فارغ وجزء حجمه (n − 1) بدل أن تنقسم إلى نصفين متقاربين. تتسلسل الاستدعاءات فيصبح عمق الاستدعاء الذاتي N = 100,000 بدل log₂N ≈ 17، ويصير مجموع المقارنات نحو N(N−1)/2 ≈ 5.0 مليار مقارنة، وهو ما يعادل عدد عمليات الخوارزميات التربيعية.

ويترتب على ذلك أيضًا أن الذاكرة المستخدمة في المكدس (Stack) ترتفع إلى O(N)، وكان الاستدعاء سيتسبب في خطأ StackOverflowError مع حجم المكدس الافتراضي، ولهذا شُغّل البرنامج في مسار بمكدس كبير.

طرق تفادي هذه المشكلة:

- اختيار المحور عشوائيًا (Random Pivot).
- اختيار وسيط ثلاثة عناصر (Median of Three): الأول والأوسط والأخير.
- اختيار العنصر الأوسط كمحور، وهو يعالج الحالتين المرتبة والعكسية.
- الاستدعاء الذاتي على الجزء الأصغر فقط وتحويل الجزء الأكبر إلى حلقة تكرارية، لضمان عمق O(log N).
- استخدام Introsort الذي ينتقل إلى Heap Sort عند تجاوز عمق محدد.

أما في الحالة المرتبة جزئيًا فقد بقي أداؤها جيدًا (6,639 µs) لأن القيم العشوائية المتفرقة تجعل العنصر الأخير غالبًا قيمة وسطية، فتبقى الأقسام متوازنة نسبيًا.

5.4 ملاحظات إضافية على النتائج

- **Bubble Sort:** نُفّذت بصيغتها الأساسية دون شرط إيقاف مبكر، لذلك استغرقت 1.58 s حتى على المصفوفة المرتبة تمامًا. ولو أضيف إليها مؤشر (Flag) يوقفها عند عدم حدوث تبديل في دورة كاملة لانخفض زمنها في هذه الحالة إلى O(N).
- **Selection Sort:** زمنها شبه ثابت في الحالات الثلاث العشوائية والمرتبة جزئيًا والمرتبة تمامًا (بين 2.18 و2.38 s)، فهي لا تتأثر بالترتيب الأولي. أما زيادة زمنها في الحالة العكسية (4.09 s) فعدد المقارنات فيها لم يتغير، ويُرجّح أن سببها سلوك المعالج مع تحديث موضع القيمة الصغرى باستمرار، وهي مسألة تتعلق بالعتاد وليست بالتعقيد النظري.
- **Insertion Sort:** زمنها في الحالة العكسية (1.24 s) أقل من العشوائية (2.07 s) رغم أن العكسية هي الحالة الأسوأ نظريًا من حيث عدد العمليات، وسبب ذلك أن حلقة الإزاحة في العكسية تتصرف بشكل متوقع تمامًا لدى المعالج بينما تتسبب البيانات العشوائية في أخطاء كثيرة في التنبؤ بالتفرع (Branch Misprediction). التعقيد النظري يبقى O(N²) في الحالتين.
- **Merge Sort:** أزمنتها متقاربة في الحالات الأربع (من 7,371 إلى 14,245 µs)، وهذا يؤكد أنها مستقرة الأداء ولا تتأثر كثيرًا بالترتيب الأولي، مقابل حاجتها إلى ذاكرة إضافية O(N).

- هذه الأزمنة من تشغيل واحد لكل خوارزمية وحالة، وقد تختلف قليلًا من تشغيل لآخر أو من جهاز لآخر، لكن الفروقات الكبيرة بين الفئات تبقى ثابتة.

6. الخلاصة

الخوارزمية	أفضل حالة	المتوسطة	أسوأ حالة	الأنسب
Selection Sort	$O(N^2)$	$O(N^2)$	$O(N^2)$	لا تتأثر بترتيب البيانات، وعدد التبديلات فيها قليل
Insertion Sort	$O(N)$	$O(N^2)$	$O(N^2)$	بيانات مرتبة أو شبه مرتبة
Bubble Sort	$O(N^2)$*	$O(N^2)$	$O(N^2)$	للتعليم فقط، وهي الأبطأ عمليًا
Merge Sort	$O(N \log N)$	$O(N \log N)$	$O(N \log N)$	أداء مضمون وثابت للأحجام الكبيرة
Quick Sort	$O(N \log N)$	$O(N \log N)$	$O(N^2)$	الأسرع عادة مع محور جيد الاختيار

* تصبح $O(N)$ إذا أُضيف شرط الإيقاف المبكر.

تثبت النتائج العملية أن التعقيد الزمني النظري ينعكس مباشرة على الأداء عند الأحجام الكبيرة، وأن الحالة الابتدائية للبيانات قد تغيّر سلوك الخوارزمية جذريًا: تستفيد Insertion Sort من الترتيب المسبق استفادة كبيرة، وتنهار Quick Sort مع محور ثابت أمام البيانات المرتبة، بينما تبقى Merge Sort ثابتة الأداء. ولذلك يجب أن يراعي اختيار الخوارزمية حجم البيانات وطبيعة ترتيبها الأولي.